

From Packing to Routing: Cross-Layer Design Space Exploration for IMC DNN Accelerators on 3D NoCs

Abstract—In-memory computing with fixed weight location shapes the traffic of NoC-based DNN accelerators: hundreds of crossbar tiles reduce partial sums through a few accumulator ports. Design becomes communication-centric, and communication is then set by three design layers — crossbar-to-tile packing, tile-to-node mapping, and packet routing — previous work optimizes one with the others held fixed. This paper explores the three as one design space on a 3D NoC across various DNN workloads. Results show that the layers couple through a metric fixed by packing, the port floor: choosing the packing for that floor rather than for traffic volume lowers mean delay $11.6\times$ and p99 $18.6\times$. Below the floor, a zero-cost mapping refinement, guided by a proposed design-time metric, recovers up to 46% of delay and 67% of p99. Adaptive selection wins in both regimes ($1.85\times/2.39\times$ above, $1.15\times/1.23\times$ below), yet leaves a residue below the floor that no offline metric predicts. Thus, packing fixes the operating point that mapping can only reach and routing can only defend. The residue routing leaves is shown to be temporal, which motivates selection that adapts in both time and space.

I. INTRODUCTION

Deep Neural Networks (DNNs) have become the dominant workload class across computing. They underpin the vast majority of AI-based applications, and DNN inference has become infrastructure, running continuously from edge devices to warehouse-scale data centers [1]. The latency and energy budget of DNN accelerators is therefore a first-order design metric, and a primary axis of competition among chip designers and manufacturers. Weight movement is what drives this budget in conventional accelerators [2], [3]. In-Memory Computing (IMC) eliminates weight traffic by keeping weights in place and performing Multiply-Accumulate (MAC) operations locally. This alleviates the weight-movement bottleneck, improving performance while reducing the communication energy budget and launching a new generation of IMC DNN accelerators [4], [5], [6]. However, no single array capacity can occupy all weights for a full DNN, making tiled architecture inevitable. For example one ResNet-50 block requires 736 crossbars (CBs) and a full network requires over ten thousand. Because ADC peripherals have a large area and power overhead, they are shared across the area of a tile. This limits the number of CBs that fit in a tile resulting in the computation being split among hundreds or even thousands of tiles connected through a Network-on-Chip (NoC) [5].

The consequence is that IMC does not eliminate communication; it relocates it. What remains mobile is activation scatter traffic and the dominant partial-sum reduction traffic: with default packing, roughly 82% of a block's NoC bytes

is reduction that converge on few accumulator tiles via the on-chip network. The accelerator's bottleneck that governs its performance and energy becomes communication, and the design focus shifts from being computation-centric to communication-centric. Communication on IMC DNN accelerators is not set by a single design choice, nor within a single design layer, but by a chain of dependent choices taken at different layers.

This paper presents a cross-layer design space exploration spanning three main layers—packing, mapping and routing—in which the design choices available at each layer are evaluated both for their performance impact within that layer and for their impact on the layers downstream. A delay- and inference-time-aware design flow is proposed, and is shown to significantly reduce both average and tail delays, the latter measured as the p99 latency that governs inference time. The contributions are:

- an attribution of communication delay to the layer that controls it: an invariant port floor fixed by the packing orientation and a peak link load fixed by the placement, with the binding term determining which layer holds control;
- identifying conditional couplings between adjacent layers: delay depends on mapping only above the packing-set floor, the selection policy's gain, including its sign is determined by mapping;
- a design flow that follows from the attribution, together with a mapping-layer design-time objective that improves delay at the knee at no communication-cost overhead;
- a bounding result: no offline quantity, over an exhaustive search, predicts which placements benefit from adaptive selection, which establishes the limit of design-time optimization and motivates online adaptivity.

Results show that the achieved improvement over the baseline reaches up to $11.6\times$ in mean delay and up to $18.6\times$ in p99 at matched load, at the packing layer; up to 46% in mean delay and 67% in p99 with zero communication-cost overhead, at the mapping layer; and up to $1.85\times$ in mean delay and $2.39\times$ in p99 over a local heuristic, at the routing and selection layer. Each gain was measured with the upstream layers already at their best, so downstream layers add to the upstream ones rather than trading against them.

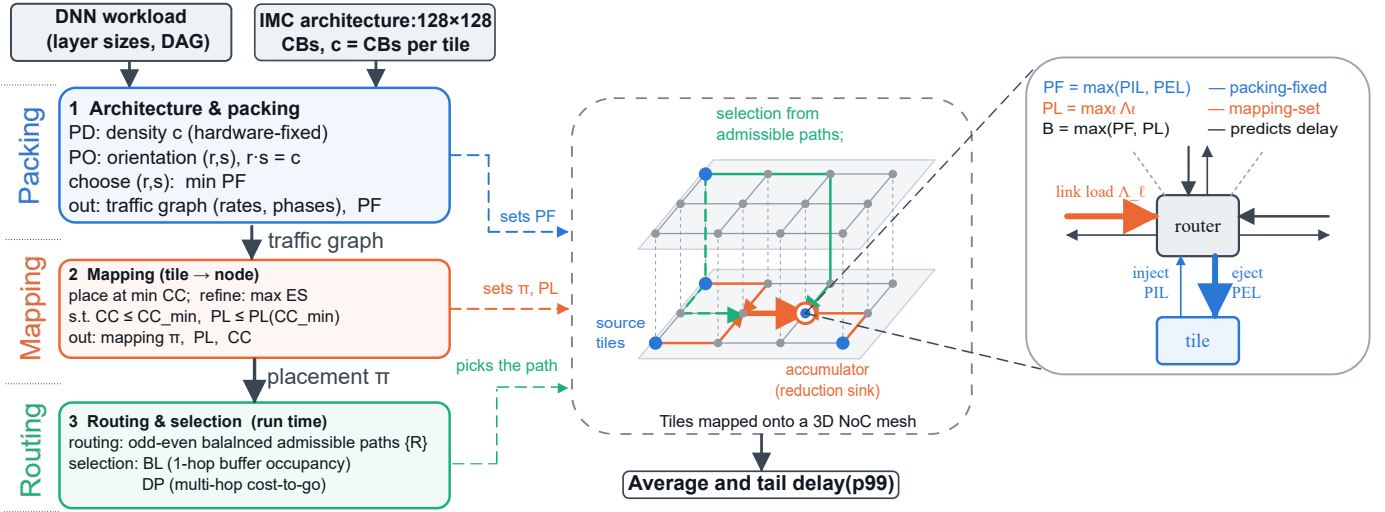


Fig. 1. The layered design flow (left), what each layer sets on the 3D NoC (center; colors match the arrows), and the ports and links of one node (right).

II. BACKGROUND AND RELATED WORK

Communication is a recognized bottleneck of DNN accelerators. Dataflow accelerators schedule data reuse so that fewer bytes move [7], [3]; IMC accelerators keep weights stationary in crossbar tiles [5], [6], [8], which relocates the remaining traffic onto the on-chip network [9].

The closest precedent is Krishnan et al. [10], who map DNN layers to tiles on a mesh and drive the NoC with per-pair injection rates, the traffic model adopted here, and conclude that the network is uncongested at average rates — a conclusion about the mean that Sec. IV-E qualifies. The same group models IMC and its NoC end to end [11] and synthesizes a NoC per DNN [12], taking topology as the variable where this work keeps a regular 3D mesh and varies routing and selection. On 3D NoCs, TTNNM [13] and TEFLON [14] optimize placement or fabric with the packing given; adaptive routing and selection [15], [16], [17] have been developed largely on synthetic traffic.

Each of these works optimizes one layer with the others held fixed: packing for utilization or traffic volume, placement for hops or heat, routing for a given placement. None asks how a choice at one layer changes what the next can achieve. The results below show that this is where the largest effects lie: the minimum-traffic packing is the wrong one, placement matters only in a regime that packing decides, and what selection yields depends on both.

III. FORMULATIONS AND METHODOLOGY

Classical NoC traffic differs dramatically from DNN traffic on a tiled IMC substrate. Most of a block's NoC traffic is partial-sum reduction, sunk by a few accumulator nodes, so that a handful of ejection ports become the hotspots of the whole network. This makes *DNN traffic a funnel, not a permutation*. Thus the binding bottleneck is a port and not a link, and it is unaffected by mapping and routing unless mapping or routing creates a hotspot link whose load exceeds the port bottleneck — an extreme case, as this study shows

and intuition supports (a hot accumulator has one ejection port but up to six incoming links).

Based on the above, the *traffic graph itself becomes a design variable, not an input*. Packing determines how much of the reduction is internalized on-tile and how much reaches the NoC, before any mapping or routing exists. Temporally, the traffic is non-stationary and bursty — layer-sequential execution gives each flow a disjoint activity window inside one inference period, so peak and average load diverge: the average decides whether a packing is feasible, while the peak decides the delay it will incur. Congestion is therefore transient and clustered at phase boundaries, on the timescale an adaptive policy's cost field need track if it is to work. Finally, the endpoint that matters is the tail rather than the mean, because reduction is a barrier in which the accumulator waits for the slowest arriving partial sum — the arrival that determines the inference time.

A. The Three Design Layers

This work proposes a cross-layer design flow decomposed into three layers (Fig. 1). Every layer is evaluated not only for its own performance but also for its downstream effect.

The top layer is *architecture and packing*. For a given hardware design it fixes how many crossbars share a tile (packing density, PD), and it decides the *layer-to-tile* mapping, i.e. how a DNN layer's rows (reduce, r) and columns (scatter, s) are grouped across those crossbars (packing orientation, PO). Together, PD and PO fix how much of a layer's traffic has to pass through a single tile port — a floor on delay that no later decision can lower. The second layer is *tile-to-node* placement, or *mapping*, which assigns NoC mesh nodes to those tiles and sets the peak single-link load across the NoC (PL). The third is the *routing and selection* policy, which decides, at run time, how much of that load can be steered off that link.

B. Packing Density, Orientation and Port Floor

A packing (c, r, s) determines how crossbars of a DNN layer are split among tiles, with $c = r \cdot s$, where r defines reduction dimension grouping, internalizing partial-sum traffic on-tile, and s defines scatter dimension grouping, internalizing input activation broadcast. Packing results in a *traffic graph* whose nodes are logical tiles and arcs are inter-tile data flows. Flow f is characterized by data rate λ_f flits/cycle from tile u_f to tile v_f and is active over the window $[t_f^{\text{on}}, t_f^{\text{off}})$ of the inference period T .

A tile's injection (outbound) and ejection (inbound) channels are different which causes tile's two port rates bind separately. The elementary intervals $\mathcal{I}$ are the maximal windows on which the active flow set is constant. Following the above formulation Peak injection (PIL) and ejection (PEL) loads and the *port floor* (PF) are

$$\text{PIL} = \max_{I \in \mathcal{I}, n \in \mathcal{N}} \sum_{f: u_f=n, f \text{ active in } I} \lambda_f, \quad (1)$$

$$\text{PEL} = \max_{I \in \mathcal{I}, n \in \mathcal{N}} \sum_{f: v_f=n, f \text{ active in } I} \lambda_f, \quad (2)$$

$$\text{PF} = \max(\text{PIL}, \text{PEL}). \quad (3)$$

where I ranges over $\mathcal{I}$ and n over the tiles of the traffic graph ($\mathcal{N}$).

These metrics are mapping-independent because equations (1)–(3) involve no placement variable, so PIL, PEL and hence PF are *placement-invariant*: remapping determines which NoC node is hot, but not how hot it is. PF is the quantity that the packing layer controls. The upper bound on the load the system can sustain in a burst is $1/\text{PF}$.

C. Mapping: Peak Link Load

A mapping is an injective map π that assigns tiles to mesh nodes. Once π is fixed, the inter-tile flows are carried on NoC links through the routing function $\mathcal{R}$. Let $\varphi_f(\ell, \pi) \in [0, 1]$ be the fraction of flow f 's minimal $\mathcal{R}$ -admissible paths that traverse directed link ℓ . An even spread over admissible paths gives the offered link load $\Lambda_\ell = \sum_f \lambda_f \varphi_f(\ell, \pi)$, again maximized over $\mathcal{I}$. The mapping layer is then characterized by peak link load (PL) and communication cost (CC) — a hop-count energy proxy

$$\text{PL} = \max_{\ell} \Lambda_\ell, \quad (4)$$

$$\text{CC} = \sum_f \lambda_f h(\pi(u_f), \pi(v_f)). \quad (5)$$

with h the Manhattan hop distance. PL and CC are determined by π ; PF is fixed at the upstream packing layer. The link attaining PL is denoted ℓ^* .

The packing and mapping layers meet at the *binding load*

$$B = \max(\text{PF}, \text{PL}), \quad (6)$$

which is a design objective optimized by the proposed flow.

D. Routing and Selection

The third layer is *routing and selection*. Routing computes the set of allowed paths $\mathcal{R}$, which binds to the mapping layer

through $\varphi_f(\ell, \pi)$. More permissive paths for a flow f help to spread it over more links, lowering PL for the same π .

Selection then determines which output, among the admissible ones, a packet is forwarded to. It is the only decision that can respond to congestion (or any run-time optimization metric) rather than predict it. However, it is bounded by what the upstream layers have left: load can be moved off ℓ^* only insofar as the flows sharing it have alternatives. Thus, mapping has an impact on what selection can achieve, provided a proper offline metric captures the escapable load. This is the key idea behind the metrics proposed in this work and described below.

E. Proposed Metrics: Escapable Load Fraction and Slope

The proposed metrics stem from the fact that *selection can only divert load that has somewhere to go*. A flow whose fraction on link ℓ is 1 ($\varphi_f(\ell) = 1$) is *forced* through ℓ . We define the *escapable fraction* for link ℓ as the share of its load carried by flows with an alternative path

$$E(\ell) = 1 - \frac{1}{\Lambda_\ell} \sum_{f: \varphi_f(\ell)=1} \lambda_f, \quad (7)$$

i.e. it is the complement of the *forced* load. We write escapable fraction of the link with peak load (PL) $E = E(\ell^*)$.

The hottest link escape, however, does not fully account for how the escape room is distributed across the congested links.

Let $\mathcal{L}_q$ be the top- q fraction of links ordered by Λ_ℓ , and let $E_q = \sum_{\ell \in \mathcal{L}_q} E(\ell) \Lambda_\ell / \sum_{\ell \in \mathcal{L}_q} \Lambda_\ell$ be the load-weighted escapable fraction of that tier. We propose the *escape slope*

$$\text{ES} = E_{q_2} - E_{q_1}, \quad (8)$$

the change in escape room between the top q_1 and the top q_2 of links ($q_2 > q_1$; the values used are set in Sec. IV-F). ES is computed offline from the logical traffic graph and the mapping.

ES stems from the observation that hot links are typically created by forced traffic, whose load cannot move, while the load queued on the links feeding them can. Thus, high ES means high escape room just below the peak, allowing backed-up load to drain sideways before it reaches the forced high-load core (the top q_1). Conversely, low ES means the shoulder is as rigid as the core and nothing drains. To improve the performance of the selection policy, ES is therefore maximized.

IV. RESULTS

A. Experimental Tools and Setup

To evaluate the proposed design flow and metrics, we employ Noxim, a cycle-accurate SystemC-based NoC simulator [18]. Noxim is a 2D NoC simulator, so we extended it to enable 3D and implemented the routing and selection used in this work which were not available in the original version: namely the balanced odd-even routing [15] and dynamic programming network selection [16].

The DP cost field reconvergence time is determined by the topology diameter and node count. For the target mesh it is 648 cycles with DP clocked at $4\times$ the network clock.

A 3D mesh NoC is adopted building on recent work in tiled DNN accelerators [13], [14]. The main motive for 3D NoC

is the rise in fan-out and fan-in, which the third dimension provides (4 links per node in 2D vs 6 in 3D) and which the funnel DNN bottleneck can best exploit. This is also confirmed by our results: confining a 36-node placement to one plane of the same mesh, against a $3 \times 4 \times 3$ block (packing (32, 4, 8), three workloads, three seeds), costs up to 40% in communication cost and 50% in peak link load, and $2.2 \times$ in p99 under congestion. Load capacity, however, is unchanged, since it is packing-dependent and placement-invariant and is set by the port floor in both geometries.

The NoC is a $6 \times 6 \times 3$ mesh of 108 tiles. A tile holds c crossbars of 128×128 1-bit cells, with 8-bit weights spread over eight bit-planes; activations are INT8 and partial sums INT32, as the accumulation depth requires.

NoC Flit size is 4 bytes, the packet is fixed at 16 flits, input buffers size is 16 flits, and the substrate runs at 1GHz clock. NoC links and tile's injection/ejection ports carry one 4-byte flit per 1 ns cycle, giving 4 GB/s per channel, with independent injection and ejection channels.

Changing the mesh simultaneously redefines both the mapping space and the routing substrate, because topology is not another layer below routing but a parallel design axis. Thus, in this work the topology is fixed to keep the layer attribution identifiable.

B. Workloads and Traffic Generation

Workloads are ResNet-50 [19], VGG-16 [20] and DeiT-S [21]. The three DNN workloads span a residual CNN, a plain CNN and a transformer encoder. They differ in partial-sum-to-activation ratio from $8 \times$ (DeiT-S) to $33 \times$ (VGG-16), and NoC bandwidth demand, by $5\text{--}6 \times$, and in structure: a deep reduction funnel with a parallel shortcut (ResNet-50), load distributed across many sinks of similar weight with no isolated funnel (DeiT-S), and a sequence of deep reductions (VGG-16). They are converted offline into statistical traffic tables the simulator can run, with per-pair injection rates and phase windows. Table I lists the three workloads. A workload inference period (compute time) is identical across its packings, and the activation and partial-sum volumes shown are per inference and packing-independent.

All runs use an appropriate warm-up and are averaged over 3 to 30 seeds, with offered load swept by a scalar multiplier k on all injection rates, where $k = 1$ is a per-workload reference scale rather than a nominal operating point. Unless stated otherwise, comparisons are made inside the knee window, defined as the load range in which mean BL delay stays below an empirically set multiple ($30 \times$) of its low-load delay, the delay at the lightest load level simulated. Because mappings differ in their saturation loads, capacity is measured per mapping as k^* : the injection scale at which a mapping's delay reaches twice its *own* low-load delay. The mapping search and the computation of design-time metrics (PF, PL, CC, ES) are done in Python from the traffic tables. Approximately 36 600 simulation runs were conducted to obtain the results below.

TABLE I
WORKLOADS, ASSUMING 128×128 CROSSBARS (CB). THE THREE BLOCKS SPAN A RESIDUAL CNN (RESNET-50), A PLAIN CNN (VGG-16) AND A TRANSFORMER ENCODER (DEiT-S).

workload	block	layers	weights (M / CBs)	period (kcyc)	act. (MB)	psum (MB)
ResNet-50	stage-3	4 (3 conv + shortcut)	1.51 / 736	38.5	0.80	9.2
DeiT-S	encoder	6 (q,k,v,o,ff1,ff2)	1.77 / 864	58.1	1.36	10.9
VGG-16	block 3	3 (all conv)	1.47 / 720	115.6	4.42	144.5

C. Design Flow and Search Procedure

To evaluate the proposed design flow, the orientation that minimizes PF at a given packing density c is selected and placed with minimum communication cost (CC) mappings. The placement is refined by maximizing ES subject to $CC \leq CC_{\min}$ and $PL \leq PL(CC_{\min})$, and adaptive selection is applied; PL is normally below the floor but the mapping and selection experiments deliberately cross it to expose that regime. Mappings are found by a seeded guarded hill-climb with 300 steps of 50 candidates each. Moves violating the constraints are rejected.

The baseline at each layer is the objective previous work optimizes: minimum traffic volume for packing, minimum communication cost CC for mapping [13], and local buffer-level selection for routing. Each layer is then evaluated against its baseline with the upper layers at their best.

D. Packing Impact

The packing layer decides (c, r, s) for a given workload. It acts before any placement exists and determines, PF, the number of tiles required, and the total inter-tile bandwidth. All can be computed offline from the traffic graph. Three density values are evaluated; 8, 16, and 32 CBs/tile. Within each density c we identify the orientation, (r, s) , minimizing total NoC traffic, the classical pre-network objective, and the one minimizing PF. Table II reports both. The two objectives coincide in four of the nine (workload, c) cells. In the remaining five, ranking on PF lowers the floor by $1.27\text{--}3.15 \times$ at a cost of $1.07\text{--}1.67 \times$ in traffic.

To determine whether a lower floor justifies its traffic overhead, so we simulate each workload's largest floor-ratio pair (min-PF and min-traffic) from Table II. To exclude the impact of mapping and keep the comparison at the packing

TABLE II
MIN-PF VS. MIN-TRAFFIC ORIENTATION PER DENSITY, WITH THE FLOOR GAINED / TRAFFIC COST OF SELECTING ON PF ("—": SAME). PF IN FLITS/CYCLE.

workload	c	min-PF		min-traffic			floor / traffic	
		(r, s)	tiles	PF	(r, s)	tiles		PF
ResNet-50	8	(2,4)	92	0.392	(2,4)	92	0.392	—
	16	(2,8)	46	0.258	(4,4)	56	0.717	$2.78\times / 1.08\times$
	32	(4,8)	28	0.327	(4,8)	28	0.327	—
DeiT-S	8	(1,8)	108	0.678	(1,8)	108	0.678	—
	16	(1,16)	66	0.905	(4,4)	66	2.849	$3.15\times / 1.62\times$
	32	(2,16)	40	0.773	(4,8)	33	1.384	$1.79\times / 1.67\times$
VGG-16	8	(4,2)	104	0.958	(4,2)	104	0.958	—
	16	(4,4)	52	0.943	(8,2)	64	1.194	$1.27\times / 1.25\times$
	32	(8,4)	32	0.760	(16,2)	40	1.953	$2.57\times / 1.07\times$

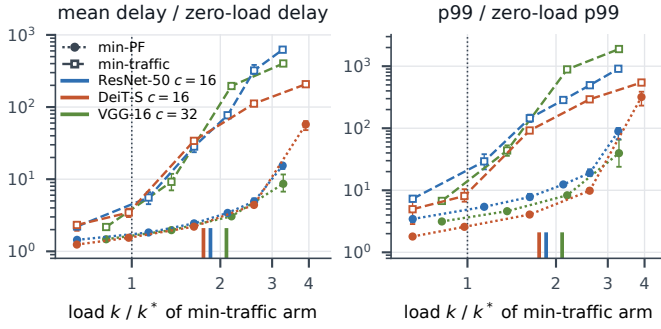


Fig. 2. Min-PF (dotted) vs. min-traffic (dashed) orientation under BL, to $3\times$ the min-traffic knee; load relative to that arm’s k^* , delay relative to zero-load delay; bars span three placements, ticks mark the min-PF arm’s k^* .

layer, both arms receive min CC placements and run under the same load levels (k) with three placements per arm and three seeds per level, under buffer-level selection (BL).

Fig. 2 plots the three sweeps mean delay and p99 (relative to the measured zero-load delay) against load (relative to the min-traffic arm’s k^*). It can be seen that all three min-traffic arms fall on one band and the three min-PF arms on a lower one. In terms of capacity, k^* is $1.75\text{--}2.10\times$ higher on the min-PF arm, and two load levels past the min-traffic knee. In terms of delay gain, the gap is $12\times$, $25\times$ and $64\times$ in mean delay and $19\times$, $30\times$ and $107\times$ in p99. All mappings stay under their floor ($PL/PF \leq 0.91$), so the gap is not a placement artifact. The same holds under DP.

The min-PF traffic overhead is therefore justified on all three workloads and *packing sets the operating point that every downstream layer inherits*.

E. Mapping Exploration

With PF fixed by packing, the mapping layer controls PL, and we ask whether moving it moves delay. For ResNet-50 at its $c=8$ minimum-PF packing (2,4), $PF = 0.392$, 24 placements are generated using the hill-climb, spanning $PL = 0.127\text{--}0.490$ ($3.9\times$), seven of them above the floor, i.e. link-bound. Each placement is simulated at $k = 1.80$, with 30 seeds each, under local selection (BL).

Fig. 3 plots mean delay against PL for those placements. Delay shows no significant response to PL until PL crosses PF and responds sharply afterward. This is confirmed by a regression with a hinge at the floor, which reaches $R^2 = 0.735$ against 0.404 for a straight line: the below-floor slope is near zero and the above-floor slope is $+655$ cycles per unit PL. These results show that delay tracks $B = \max(PF, PL)$, and which of its two terms is active depends on whether the placement has crossed the floor.

Hop count acts separately but weakly. For the same B below the floor, 15% more communication cost causes only a 3.7–6.7% increase in low-load delay.

Placement below the floor still significantly impacts delay near the knee ($13.6\times$ across eight placements of ResNet-50 (8,1,8) (see Fig. 5), against $1.3\times$ at light load or saturation). The min CC placement, however, is shown to capture most of it. Searching beyond min CC returns at most $1.14\times$. The

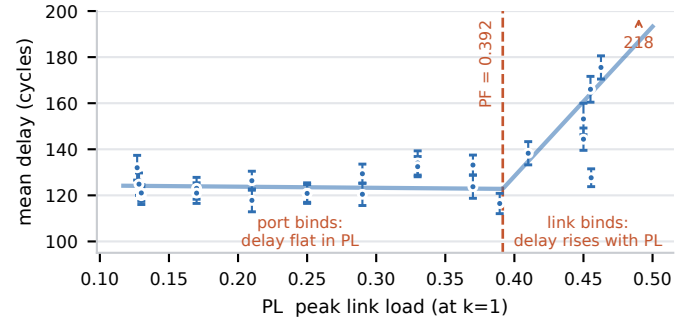


Fig. 3. Mean delay vs. PL, 24 placements of ResNet-50 (8,2,4) at $k = 1.80$ under BL, hinge fit at $PF = 0.392$.

picture flips above the floor. Across five populations on the three workloads ($n = 8\text{--}24$ each), delay spreads widely ($4.6\text{--}31.8\times$) between best and worst placement. That spread is ordered by PL/PF in every population. The VGG-16 and DeiT-S workloads show the same trend: delay is nearly flat in PL below the floor and rises with it above.

F. Mapping Refinement

For the below-floor mapping at its min-CC, we investigate whether there is a room for further refinement through maximizing the proposed metric ES with CC and PL held constant. Escape room varies with PL and communication cost, so we tested it by intervention. The packing and the seed are fixed and search start from the min-CC placement, and run two guarded climbs from it, one minimizing and one maximizing $ES = E_{20} - E_5$, both constrained to $CC \leq CC_{\min}$ and $PL \leq PL(CC_{\min})$. The $5 \rightarrow 20$ span is the best of 18 endpoint pairs screened against measured gain on nine packings. This is done on three arms, eight seeds each (Fig. 4).

Fig. 4 shows the result. It can be seen that maximizing ES lowers mean delay by up to $1.25\times$ (20%) and p99 by up to $1.43\times$ (30%) at the arm level, and by up to $1.85\times$ and $2.99\times$ (46% and 67%) on the best placement with no CC or PL overheads. The gain appears under BL on two arms and under DP on the third, which confirms it is a property of the placement, not of the selection.

This lever has no effect on dominant-flow traffic, as expected. For example, on VGG-16 (32,4,8), where one pair carries 92% of the peak link, maxES returns nothing. This can be predicted by whether the climb can move ES.

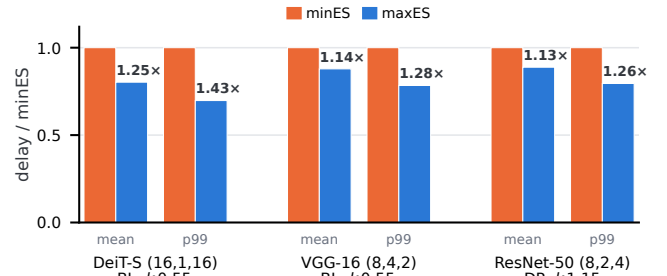


Fig. 4. maxES vs. minES from the same min-CC start at matched CC and PL; eight seeds per arm at the knee load and policy shown ($p = 0.016$ on p99, three arms combined). Bars: $1/\text{mean paired ratio}$; labels: the ratio.

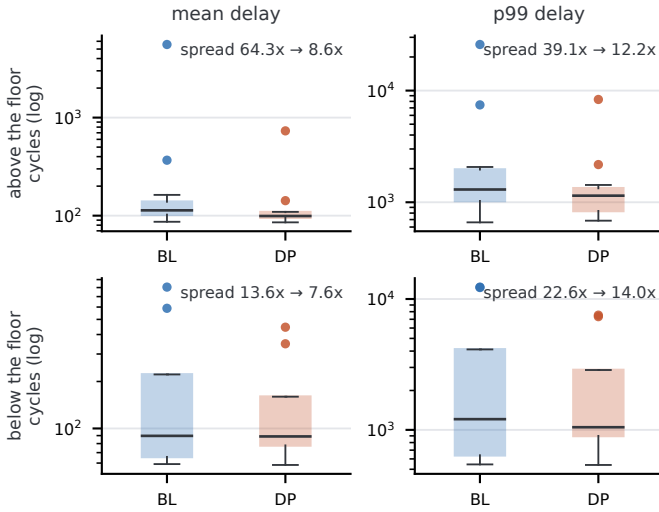


Fig. 5. Delay across placements under BL and DP: 12 equal-PL placements of ResNet-50 (16, 2, 8) above the floor (top), 8 of (8, 1, 8) at the knee below (bottom).

G. Routing and Selection

The last layer acts at run time. With packing and placement fixed, selection decides which of the admissible outputs a packet takes, and we compare the local buffer-level policy (BL) used so far against the distributed cost-to-go policy (DP) on identical placements and seeds, so that the policy is the only difference between paired runs.

DP’s effect is on the *spread* of delay across placements, the mapping layer’s main risk. DP narrows the spread in all eight populations in which BL exhibits one — three workloads, both regimes, $n = 7$ –24 placements at three seeds, read inside each policy’s knee window — by $1.09\times$ to $7.50\times$; Fig. 5 shows one per regime. The mechanism is rescue of the worst placements rather than acceleration of the best: on the fastest placement of each population BL matches or beats DP, while on the slowest DP improves delay by up to $7.6\times$ (5 566 to 732 cycles).

H. Policy Choice and Its Limit

We ask which fixed policy is better over everything measured, and how much a policy that could switch per operating point would add. Every knee-window placement $\times$ load cell from the mapping and routing populations of Secs. IV-E–IV-G is used — 701 above the floor, 128 below, three seeds each — and three policies are compared on identical cells: always-BL, always-DP, and a per-cell oracle taking the better of the two.

DP is ahead of BL in both regimes, on 65 of 71 above-floor placements and in 61 of the 62 load bands measured: pooled over cells, by $1.85\times$ in mean delay and $2.39\times$ in p99 above the floor, and by $1.15\times$ and $1.23\times$ below it (the blue dotted lines in the routing panel of Fig. 6). The same panel resolves this per workload: 1.62 – $3.10\times$ above the floor on all three, 1.15 – $1.26\times$ below it on ResNet-50 and VGG-16, and level ($1.01\times$) on DeiT-S below the floor — the arm that Fig. 4 accordingly reads under BL. The oracle (the grey lines of the same panel) adds 1.1% and 2.5% above the floor but 3.8% and 7.7% below it: of the improvement available between always-

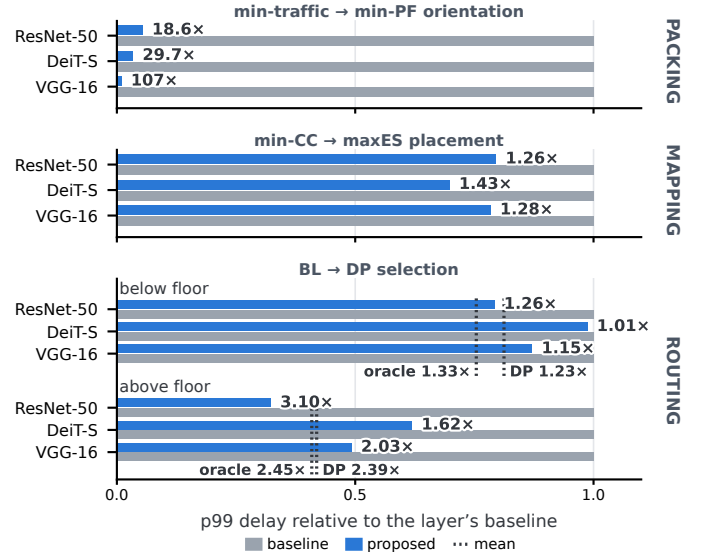


Fig. 6. p99 gain at each layer against its own baseline (grey), per workload; routing by regime, with the pooled DP gain and the per-cell BL/DP oracle over each regime’s population dotted.

BL and the oracle, always-DP captures 99% and 98% above the floor but only 81% and 76% below it.

The remaining 4–8% below the floor is the only headroom left, so we ask whether the cells DP loses can be identified at design time: no offline quantity survives a held-out packing, and a pre-registered model over 15 features of 56 placements reaches $R^2 = -0.03$ on DP’s gain when a packing is held out, against a registered ceiling of 0.10.

Taken together in Fig. 6, each layer read against its own baseline at its own operating point, packing carries the gain (18.6 – $107\times$), mapping adds 1.26 – $1.43\times$, and selection adds what the regime allows. Mean delay gives the same picture at smaller magnitudes: the layer ordering is unchanged, and the oracle sits 1.1% from DP above the floor and 3.8% below it.

V. CONCLUSION

This paper presents a cross-layer design flow for IMC DNN accelerators on a 3D NoC. Prior studies design for a single layer, which limits design freedom and forgoes the gains shown here. The proposed flow is evaluated on three workloads. The design layers are found to be highly coupled. The first layer (packing), which decides the port floor, is shown to have the highest impact: the optimal design choice for delay lowers mean delay $11.6\times$ and p99 $18.6\times$ at matched load. Below that floor, mapping can be optimized through a proposed metric (escape slope); around the knee, at no communication cost, this metric is shown to recover up to 46% of mean delay and 67% of p99. At the selection layer, adaptive selection is the better fixed policy in both regimes, by $1.85\times/2.39\times$ above the floor and $1.15\times/1.23\times$ below it.

Always-DP is shown to capture 98–99% of the improvement an oracle switching policy per placement and load would achieve above the floor and 76–81% below it. No offline quantity predicts which cells it loses, because the residue is temporal: operating below the floor produces transient

congestion clustered at phase boundaries, on a timescale a spatially adaptive policy cannot track. Closing this residue motivates a temporally as well as spatially adaptive selection policy. The same design flow admits other objectives, such as thermal-aware placement and load balancing.

REFERENCES

- [1] N. P. Jouppi *et al.*, “In-Datacenter Performance Analysis of a Tensor Processing Unit,” in *Proceedings of the 44th Annual International Symposium on Computer Architecture (ISCA)*, vol. 45. New York, NY, USA: Association for Computing Machinery, Jun. 2017, pp. 1–12.
- [2] M. Horowitz, “1.1 Computing’s energy problem (and what we can do about it),” in *2014 IEEE International Solid-State Circuits Conference Digest of Technical Papers (ISSCC)*, 2014, pp. 10–14.
- [3] V. Sze, Y.-H. Chen, T.-J. Yang, and J. S. Emer, “Efficient Processing of Deep Neural Networks: A Tutorial and Survey,” *Proceedings of the IEEE*, vol. 105, no. 12, pp. 2295–2329, 2017.
- [4] A. Sebastian, M. Le Gallo, R. Khaddam-Aljameh, and E. Eleftheriou, “Memory devices and applications for in-memory computing,” *Nature nanotechnology*, vol. 15, no. 7, pp. 529–544, 2020.
- [5] A. Shafiee, A. Nag, N. Muralimanohar, R. Balasubramonian, J. P. Strachan, M. Hu, R. S. Williams, and V. Srikumar, “ISAAC: A Convolutional Neural Network Accelerator with In-Situ Analog Arithmetic in Crossbars,” in *2016 ACM/IEEE 43rd Annual International Symposium on Computer Architecture (ISCA)*, 2016, pp. 14–26.
- [6] P. Chi, S. Li, C. Xu, T. Zhang, J. Zhao, Y. Liu, Y. Wang, and Y. Xie, “PRIME: A Novel Processing-in-Memory Architecture for Neural Network Computation in ReRAM-Based Main Memory,” in *2016 ACM/IEEE 43rd Annual International Symposium on Computer Architecture (ISCA)*, 2016, pp. 27–39.
- [7] Y.-H. Chen, J. Emer, and V. Sze, “Eyeriss: A Spatial Architecture for Energy-Efficient Dataflow for Convolutional Neural Networks,” in *2016 ACM/IEEE 43rd Annual International Symposium on Computer Architecture (ISCA)*, 2016, pp. 367–379.
- [8] Y. Long, D. Kim, E. Lee, P. Saha, B. A. Mudassar, X. She, A. I. Khan, and S. Mukhopadhyay, “A Ferroelectric FET-Based Processing-in-Memory Architecture for DNN Acceleration,” *IEEE Journal on Exploratory Solid-State Computational Devices and Circuits*, vol. 5, no. 2, pp. 113–122, 2019.
- [9] S. M. Nabavinejad, M. Baharloo, K.-C. Chen, M. Palesi, T. Kogel, and M. Ebrahimi, “An Overview of Efficient Interconnection Networks for Deep Neural Network Accelerators,” *IEEE Journal on Emerging and Selected Topics in Circuits and Systems*, vol. 10, no. 3, pp. 268–282, Sep. 2020.
- [10] G. Krishnan, S. K. Mandal, C. Chakrabarti, J.-s. Seo, U. Y. Ogras, and Y. Cao, “Impact of On-chip Interconnect on In-memory Acceleration of Deep Neural Networks,” *ACM Journal on Emerging Technologies in Computing Systems*, vol. 18, no. 2, Apr. 2022.
- [11] G. Krishnan, S. K. Mandal, M. Pannala, C. Chakrabarti, J.-S. Seo, U. Y. Ogras, and Y. Cao, “SIAM: Chiplet-based Scalable In-Memory Acceleration with Mesh for Deep Neural Networks,” *ACM Transactions on Embedded Computing Systems (TECS)*, vol. 20, no. 5s, pp. 68:1–68:24, Sep. 2021.
- [12] S. K. Mandal, G. Krishnan, C. Chakrabarti, J.-S. Seo, Y. Cao, and U. Y. Ogras, “A Latency-Optimized Reconfigurable NoC for In-Memory Acceleration of DNNs,” *IEEE Journal on Emerging and Selected Topics in Circuits and Systems*, vol. 10, no. 3, pp. 362–375, Sep. 2020.
- [13] X. Li, W. Fan, H. Zhang, J. Ji, T. Cheng, S. Li, L. Li, and Y. Fu, “TTNNM: Thermal- and Traffic-Aware Neural Network Mapping on 3D-NoC-based Accelerator,” in *Proceedings of the Great Lakes Symposium on VLSI 2024*, ser. GLSVLSI ’24. Clearwater, FL, USA: Association for Computing Machinery, 2024, pp. 364–369.
- [14] G. Narang, C. Ogbogu, J. R. Doppa, and P. P. Pande, “TEFLON: Thermally Efficient Dataflow-aware 3D NoC for Accelerating CNN Inference on Manycore PIM Architectures,” *ACM Trans. Embed. Comput. Syst.*, vol. 23, no. 5, Aug. 2024.
- [15] N. Dahir, T. Mak, F. Xia, and A. Yakovlev, “Highly Adaptive and Deadlock-Free Routing for Three-Dimensional Networks-on-Chip,” *IET Computers & Digital Techniques*, vol. 7, no. 6, pp. 255–263, 2013.
- [16] T. Mak, P. Y. Cheung, W. Luk, and K. P. Lam, “A DP-network for optimal dynamic routing in network-on-chip,” in *Proceedings of the 7th IEEE/ACM International Conference on Hardware/Software Codesign and System Synthesis*, ser. CODES+ISSS ’09. Grenoble, France: Association for Computing Machinery, 2009, pp. 119–128.
- [17] W. Choi, K. Duraisamy, R. G. Kim, J. R. Doppa, P. P. Pande, D. Marculescu, and R. Marculescu, “On-Chip Communication Network for Efficient Training of Deep Convolutional Networks on Heterogeneous Manycore Systems,” *IEEE Transactions on Computers*, vol. 67, no. 5, pp. 672–686, May 2018.

- [18] V. Catania, A. Mineo, S. Monteleone, M. Palesi, and D. Patti, "Cycle-Accurate Network on Chip Simulation with Noxim," *ACM Transactions on Modeling and Computer Simulation (TOMACS)*, vol. 27, no. 1, pp. 4:1–4:25, Aug. 2016.
- [19] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Jun. 2016, pp. 770–778.
- [20] K. Simonyan and A. Zisserman, "Very Deep Convolutional Networks for Large-Scale Image Recognition," *CoRR*, Sep. 2014.
- [21] H. Touvron, M. Cord, M. Douze, F. Massa, A. Sablayrolles, and H. Jégou, "Training data-efficient image transformers & distillation through attention," in *International Conference on Machine Learning*, Dec. 2020.